

Cours n°11

Programme du cours

Activité	Durée
Création d'un projet ASP.NET Core MVC	15m
Création d'une base de données	15m
Page de nouveau compte	45m
Page de connexion	30m
Page utilisateur	15m
Gestion des sessions	45m
Page de mise à jour	30m
Page des personnes inscrites	30m
Hashing des mots de passes	30m

Objectif

Gérer la connexion des utilisateurs

L'utilisateur aura la possibilité de s'inscrire, de se connecter, de mettre à jour son login et son mot de passe et de supprimer son compte

Installation

- Créer un projet ASP.NET Core MVC
- Ajouter une base de données, elle contient une table « User » avec une colonne « Id », « Login », « Password »
- ID doit être une clé primaire avec un auto-incrément. Login et Password sont des champs NVARCHAR(MAX). Aucune colonne n'est nullable.

Page de nouveau compte

Faire une Page pour que l'utilisateur puisse s'inscrire. Le formulaire est composé d'un champ *Login* et d'un champ *Password*.

Il faut empêcher l'ajout d'une personne si le login existe déjà et afficher un message d'erreur

Création d'un nouveau compte

Login

Mot de passe

Page de connexion

Modifier le fichier Home/Index.html pour y ajouter les champs Login, Password et un lien pour la connexion et l'inscription

Connexion

Login

Mot de passe

[Créer un nouveau compte](#) [Afficher tous les utilisateurs](#)

Page utilisateur

Ajout d'une Page quand l'utilisateur est connecté

Afficher « Bonjour [login] »

Ajouter un lien pour supprimer ou modifier le compte

Interface utilisateur

Bonjour eric

[Supprimer mon Compte](#) [Modifier mon Compte](#)

Gestion des sessions

Explications

Dans une application web, le protocole HTTP est *stateless*, ce qui signifie que le serveur oublie complètement l'utilisateur entre chaque requête. Sans mécanisme d'authentification, chaque page ou action serait considérée comme provenant d'un utilisateur inconnu. L'objectif de l'authentification est donc de permettre au serveur de reconnaître un utilisateur entre plusieurs requêtes et de savoir qui est connecté.

Lorsqu'un utilisateur soumet le formulaire de connexion, ASP.NET Core crée un Claim contenant son nom d'utilisateur. Ce Claim est placé dans un ClaimsIdentity, puis dans un ClaimsPrincipal, qui représente l'utilisateur connecté dans l'application. La méthode `HttpContext.SignInAsync()` crée alors un cookie d'authentification envoyé au navigateur. À chaque requête suivante, le navigateur renvoie automatiquement ce cookie au serveur, ce qui permet à ASP.NET Core de reconnaître l'utilisateur comme étant connecté sans qu'il ait besoin de se reconnecter. Les actions protégées avec l'attribut `[Authorize]` deviennent alors accessibles uniquement aux utilisateurs authentifiés.

Prérequis

Dans `program.cs` en dessous de

```
// Add services to the container.  
builder.Services.AddControllersWithViews();
```

Ajouter la configuration pour stocker la connexion de l'utilisateur dans un cookie (ajouter les using si nécessaire)

```
builder.Services  
    .AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)  
    .AddCookie(options =>  
    {  
        options.LoginPath = "/";  
        options.AccessDeniedPath = "/";  
    });
```

En dessus de

```
app.UseAuthorization();
```

Ajouter

```
app.UseAuthentication();
```

Login

Dans votre méthode de login ajouter le code suivant :

```
var claims = new List<Claim>
{
    new Claim(ClaimTypes.Name, username)
};

var identity = new ClaimsIdentity(
    claims,
    CookieAuthenticationDefaults.AuthenticationScheme);

var principal = new ClaimsPrincipal(identity);

await HttpContext.SignInAsync(
    CookieAuthenticationDefaults.AuthenticationScheme,
    principal);
```

L'utilisateur est maintenant connecté, pour protéger un endpoint il faut mettre l'attribut [Authorize] au-dessus comme ceci :

```
[Authorize]
0 références
public IActionResult MonEndpointProteger()
{
    ...
}
```

Page de mise à jour

Faire une Page permettant de modifier le login et le mot de passe

Mise à jour du compte

Nouveau login

Nouveau mot de passe

Mettre à jour

Page des personnes inscrites

Ajouter une page qui affiche tous les utilisateurs inscrits

Hashing des mots de passes

Il faut implémenter une méthode de hashing pour le champ **Password**